

Informe Técnico: Sistema de Archivos Distribuido Basado en Etiquetas (TBFS)

1. Arquitectura

Organización del Sistema Distribuido

El sistema TBFS está organizado en una arquitectura de tres capas principales:

Capa de Descubrimiento (Registry Service): - 3 nodos replicados que mantienen un catálogo de servicios activos - Implementa consenso distribuido tipo Raft para garantizar consistencia - Detecta y elimina servicios inactivos automáticamente

Capa de Metadatos (MetaNameNode): - 3 réplicas que gestionan exclusivamente metadatos (nombres, etiquetas, hashes) - No almacena archivos físicos, solo información sobre ellos - Coordina la asignación de réplicas a DataNodes

Capa de Almacenamiento (DataNodes): - Múltiples nodos (5 por defecto) que almacenan archivos físicos - Cada archivo se replica en 3 DataNodes diferentes - Operan de forma autónoma pero coordinados por el MetaNameNode

Roles del Sistema

Registry Service: - Mantiene registro distribuido de todos los servicios (MetaNameNodes y DataNodes) - Proporciona descubrimiento dinámico de servicios - Implementa elección de líder mediante votación

MetaNameNode: - Gestiona catálogo de metadatos (archivos, etiquetas, ubicación de réplicas) - Asigna réplicas a DataNodes según espacio disponible y carga - Procesa operaciones de archivos (crear, leer, eliminar, buscar) - Implementa elección de líder para operaciones de escritura

DataNodes: - Almacenan archivos físicos en sistemas de archivos locales - Reportan estado periódicamente (heartbeats) al MetaNameNode - Responden a solicitudes de lectura/escritura del MetaNameNode

Cliente: - Frontend web (Streamlit) e interfaz CLI - Consulta Registry para descubrir servicios - Se comunica con MetaNameNode líder para operaciones

Distribución de Servicios en Red Docker Swarm

Red Docker Swarm (tbfs_net): - Red overlay de Docker Swarm que conecta todos los servicios - Permite comunicación mediante nombres de servicio Docker - Red distribuida que funciona en múltiples nodos del swarm - Resolución de nombres automática entre servicios

Registry Cluster: - 3 servicios desplegados en el swarm: `tbfs-registry-1`, `tbfs-registry-2`, `tbfs-registry-3` - Puertos expuestos: 9000, 9001, 9002 -

Comunicación interna mediante nombres de servicio del swarm - Pueden ejecutarse en diferentes nodos del swarm

MetaNameNode Cluster: - 3 servicios desplegados en el swarm: **tbfs-namenode-1**, **tbfs-namenode-2**, **tbfs-namenode-3** - Puertos expuestos: 8010, 8011, 8012 - Volúmenes persistentes para bases de datos SQLite - Distribuidos en nodos del swarm para alta disponibilidad

DataNodes: - 5 servicios desplegados en el swarm: **tbfs-datanode-1** a **tbfs-datanode-5** - Puertos expuestos: 8001-8005 - Volúmenes persistentes para almacenamiento de archivos - Escalables horizontalmente mediante adición de más servicios

Frontend: - 1 servicio desplegado en el swarm: **tbfs-frontend** - Puerto expuesto: 8501 - Acceso a red overlay para comunicación con backend

2. Procesos

Tipos de Procesos

Procesos de Registry: - Cada nodo ejecuta un proceso FastAPI independiente - Mantiene registro en memoria de servicios activos - Implementa hilos para: elección de líder, heartbeats, limpieza de inactivos

Procesos de MetaNameNode: - Cada réplica ejecuta un proceso FastAPI independiente - Gestiona metadatos en bases de datos SQLite locales - Implementa hilos para: elección de líder, heartbeats, monitoreo de DataNodes

Procesos de DataNode: - Cada DataNode ejecuta un proceso FastAPI independiente - Gestiona almacenamiento local de archivos - Implementa hilos para: registro automático, heartbeats periódicos

Procesos de Cliente: - Frontend: proceso Streamlit que proporciona interfaz web - CLI: scripts Python que interactúan directamente con la API

Organización de Procesos

Agrupación por Instancia: - Cada servicio está encapsulado en un contenedor Docker independiente - Procesos relacionados (múltiples réplicas) se ejecutan en instancias separadas pero idénticas - Desplegados en Docker Swarm, permitiendo distribución en múltiples nodos - Permite escalabilidad horizontal mediante adición de instancias o nodos al swarm

Arquitectura de Microservicios: - Cada componente es un servicio independiente con su propia base de datos/almacenamiento - Desarrollo y despliegue independiente - Escalabilidad selectiva según necesidades - Tolerancia a fallos parciales

Patrones de Diseño de Desempeño

Modelo de Hilos (Threading): - Hilos de monitoreo para detectar nodos inactivos - Hilos de heartbeat para mantener comunicación periódica - Hilos de elección de líder para procesos de consenso - Hilos de re-replicación automática en segundo plano

Modelo Asíncrono: - Operaciones I/O asíncronas en peticiones HTTP - Operaciones de lectura/escritura de archivos asíncronas - Operaciones de base de datos con manejo asíncrono cuando es posible

Procesos Independientes: - Cada servicio como proceso independiente - Aprovechamiento de múltiples núcleos de CPU - Aislamiento de fallos - Escalabilidad horizontal

Patrón Líder-Seguidor: - Registry y MetaNameNode implementan líder-seguidor - Líder procesa todas las operaciones de escritura - Seguidores replican operaciones del líder - Elección automática de nuevo líder ante fallo

3. Comunicación

Tipo de Comunicación

REST (Representational State Transfer): - Protocolo principal: HTTP/HTTPS - Serialización: JSON para intercambio de datos - Framework: FastAPI para todas las APIs - Validación: Modelos Pydantic para datos

Ventajas: - Simplicidad de implementación y depuración - Compatibilidad con herramientas estándar - Independencia de lenguaje y plataforma - Documentación automática de APIs

Comunicación Cliente-Servidor

Cliente → Registry: - Peticiones HTTP GET para obtener listas de servicios activos - Solo lectura, sin modificación de estado - Failover automático entre múltiples nodos del Registry

Cliente → MetaNameNode: - POST: subir archivos con etiquetas - GET: listar archivos, descargar archivos - DELETE: eliminar archivos por etiquetas - POST: agregar/eliminar etiquetas - Redirección HTTP 307 automática hacia el líder si el cliente se conecta a un seguidor

Cliente → DataNode: - No hay comunicación directa - Toda la comunicación pasa a través del MetaNameNode

Comunicación Servidor-Servidor

Registry Registry: - POST /internal/vote: solicitud de votos para elección de líder - POST /internal/replicate: replicación de registros de servicios - Heartbeats periódicos para mantener coherencia

MetaNameNode **MetaNameNode:** - POST `/internal/vote`: votación en elecciones de líder - POST `/internal/replicate`: replicación de operaciones de metadatos - POST `/internal/heartbeat`: heartbeats del líder a seguidores

MetaNameNode → **DataNode:** - POST `/datanodes/register`: registro de nuevos DataNodes - POST `/datanodes/{id}/heartbeat`: recepción de heartbeats - POST `/store`: envío de archivos para almacenamiento - GET `/retrieve/{hash}`: solicitud de lectura de archivos - DELETE `/delete/{hash}`: eliminación de archivos

DataNode → **Registry:** - Consulta Registry para descubrir MetaNameNode líder - No se registra directamente en Registry (se registra en MetaNameNode)

Comunicación entre Procesos

Intra-proceso (dentro del mismo servicio): - Variables de estado compartidas protegidas por locks (mutex) - Threading para operaciones concurrentes - Locks para acceso exclusivo a bases de datos

Inter-proceso (entre servicios): - Exclusivamente mediante peticiones HTTP REST - Desacoplamiento entre servicios - Independencia de implementación - Facilidad de escalabilidad

4. Coordinación

Sincronización de Acciones

Sincronización de Operaciones de Escritura: - Todas las escrituras procesadas por el MetaNameNode líder - Nodos no-líder redirigen automáticamente al líder - Garantiza que solo un nodo procese modificaciones en un momento dado

Sincronización de Replicación: - Líder replica operación a todos los seguidores antes de confirmar - Requiere mayoría (quorum) para confirmar operación - Garantiza consistencia entre réplicas

Sincronización de Heartbeats: - DataNodes envían heartbeats periódicos (cada 10 segundos) - MetaNameNode actualiza estado de DataNodes - Detecta fallos por ausencia de heartbeats (timeout: 30 segundos)

Sincronización de Elección de Líder: - Cooldown de 5 segundos entre elecciones para evitar elecciones frecuentes - Términos incrementales para garantizar orden - Votación requiere mayoría de nodos

Acceso Exclusivo a Recursos

Protección de Bases de Datos: - Locks (mutex) para acceso exclusivo a SQLite - Prevención de condiciones de carrera en operaciones concurrentes - Transacciones atómicas para operaciones complejas

Protección del Estado del Cluster: - Locks para estado del cluster (líder, términos, peers) - Actualizaciones atómicas del estado - Timeouts en adquisición de locks para evitar deadlocks

Gestión de Locks: - Locks de lectura-escritura donde sea apropiado - Múltiples lecturas concurrentes permitidas - Escrituras bloquean todas las operaciones

Toma de Decisiones Distribuidas

Elección de Líder (Leader Election): - Algoritmo tipo Raft implementado - Nodo detecta ausencia de líder e inicia elección - Votación basada en términos y prioridad - Candidato con mayoría de votos se convierte en líder - Líder mantiene posición mediante heartbeats periódicos

Consenso en Replicación: - Líder propone operación - Seguidores confirman recepción - Operación confirmada cuando mayoría confirma - Sistema continúa operando con mayoría restante si una réplica falla

Decisión de Asignación de Réplicas: - MetaNameNode evalúa estado de todos los DataNodes - Considera: espacio disponible, carga, estado de salud - Asigna réplicas balanceadas para distribuir carga - Excluye DataNodes en drenaje o inactivos - Algoritmo basado en hash del archivo para distribución determinística

5. Nombrado y Localización

Identificación de Datos y Servicios

Identificación de Archivos: - **Nombre original:** proporcionado por el usuario - **Hash SHA-256:** identificador único derivado del contenido - **ID numérico:** identificador único en base de datos de metadatos - El hash garantiza deduplicación y verificación de integridad

Identificación de Servicios: - **NODE_ID:** identificador único de cada instancia (ej: "namenode-1", "datanode-3") - **Nombre de servicio Docker:** nombre canónico para red (ej: "tbfs-namenode-1") - **URL completa:** protocolo + host + puerto (ej: "http://tbfs-namenode-1:8010")

Identificación de DataNodes: - Identificador único persistente a través de reinicios - Permite rastrear qué archivos están almacenados en cada DataNode

Ubicación de Datos y Servicios

Ubicación de Metadatos: - Bases de datos SQLite locales en cada réplica del MetaNameNode - Ruta: `/app/namenode/data/{node_id}/namenode.db` - Cada réplica mantiene copia completa de metadatos

Ubicación de Archivos Físicos: - Sistemas de archivos locales de los DataNodes - Ruta base: `/app/storage` (configurable) - Estructura: organizados por

primeros 2 caracteres del hash (ej: `ab/abc123...`) - Facilita distribución de carga y búsqueda

Ubicación de Servicios: - Registry Service: catálogo centralizado de servicios activos - Nombres de servicio Docker Swarm: resolución de nombres en red overlay - Puertos expuestos: acceso desde host para clientes externos - Servicios pueden ejecutarse en cualquier nodo del swarm

Localización de Datos y Servicios

Descubrimiento de Servicios: - Clientes consultan Registry: `GET /servers/active` - Registry retorna información sobre servicios activos (URLs, estado) - Servicios se registran automáticamente al iniciar - Failover automático entre múltiples nodos del Registry

Localización del Líder: - Clientes consultan cualquier réplica del MetaNameNode: `GET /` - Cada réplica responde con información del líder actual (`leader_url`) - Clientes se redirigen automáticamente al líder para escrituras - Redirección HTTP 307 (Temporary Redirect)

Localización de Réplicas de Archivos: - MetaNameNode mantiene registro en tabla `file_replicas` - Consulta: `SELECT datanode_id, replica_type FROM file_replicas WHERE file_id = ?` - Retorna lista de DataNodes ordenada por prioridad (primary, secondary, tertiary) - Sistema intenta leer desde primario, con fallback a secundarias

Búsqueda por Etiquetas: - Metadatos incluyen asociaciones archivo-etiqueta en tabla `file_tags` - Consultas SQL filtran archivos que contienen TODAS las etiquetas especificadas - MetaNameNode procesa consultas localmente en su base de datos - Query: `SELECT files WHERE tags IN (...) GROUP BY file_id HAVING COUNT(DISTINCT tag) = ?`

6. Consistencia y Replicación

Distribución de Datos

Distribución de Metadatos: - Replicación completa en todas las réplicas del MetaNameNode (3 réplicas) - Cada réplica mantiene copia completa de la base de datos - Cualquier réplica puede responder a consultas de lectura

Distribución de Archivos Físicos: - Distribución entre múltiples DataNodes mediante esquema de réplicas - Cada archivo almacenado en exactamente 3 DataNodes diferentes - Asignación basada en hash del archivo para distribución determinística

Estrategia de Distribución: - Considera: distribución equitativa de carga, espacio disponible, estado de salud - Excluye DataNodes en proceso de drenaje - Prioriza DataNodes con más espacio libre

Replicación

Réplicas de Metadatos: - 3 réplicas (una en cada MetaNameNode) - Operaciones de escritura replicadas a todas las réplicas antes de confirmarse - Consistencia fuerte: todas las réplicas deben confirmar

Réplicas de Archivos: - Cada archivo replicado en 3 DataNodes diferentes - Garantía: al menos 2 de 3 réplicas se almacenan exitosamente - Tolerancia a fallos de 1 DataNode sin pérdida de datos

Réplicas del Registry: - 3 réplicas que sincronizan estado mediante consenso distribuido - Visión consistente de servicios registrados en todos los nodos

Confiabilidad de Réplicas Tras Actualización

Modelo de Consistencia para Metadatos: - Consistencia fuerte - Todas las réplicas del MetaNameNode deben confirmar operación antes de completarse - Si una réplica falla durante operación, se aborta y se reintenta - Clientes siempre leen de réplica actualizada

Modelo de Replicación de Archivos: - Consistencia eventual con garantías de disponibilidad - Requiere que al menos 2 de 3 réplicas se almacenen exitosamente - Si una réplica falla durante escritura, sistema continúa con réplicas restantes - Réplicas fallidas se re-repican automáticamente cuando se detecta fallo

Verificación de Integridad: - Archivos identificados mediante hash SHA-256 - Permite: verificación de integridad, detección de corrupción, deduplicación - Hash calculado al subir archivo y almacenado en metadatos

Re-replicación Automática: - Monitoreo continuo del estado de DataNodes (cada 30 segundos) <- Se tiene pensado disminuir este tiempo. - Al detectar DataNode inactivo: 1. Se identifican todos los archivos afectados 2. Se lee cada archivo desde réplica activa 3. Se re-replica a nuevo DataNode disponible 4. Se actualiza registro de réplicas en MetaNameNode

7. Tolerancia a Fallos

Respuesta a Errores

Detección de Fallos: - **Heartbeats:** señales periódicas de vida (cada 10 segundos) - **Timeouts:** peticiones HTTP con timeouts configurados (3-30 segundos) - **Verificación de salud:** endpoints `/health` y `/` para verificar estado - **Ausencia de heartbeats:** indica fallo potencial (timeout: 30 segundos)

Manejo de Fallos de MetaNameNode: - Seguidores detectan ausencia de heartbeats del líder - Se inicia automáticamente elección de líder - Nuevo líder asume control y continúa procesando operaciones - Clientes se redirigen automáticamente al nuevo líder - Tiempo de recuperación: ~15-30 segundos

Manejo de Fallos de DataNode: - MetaNameNode detecta DataNodes inactivos por ausencia de heartbeats - Se identifican todos los archivos almacenados en DataNode fallido - Se inicia automáticamente proceso de re-replicación - Archivos re-replicados desde réplicas activas a nuevos DataNodes - Proceso en segundo plano, no bloquea operaciones normales

Manejo de Fallos de Registry: - Si líder del Registry falla, se realiza elección de líder - Nuevo líder asume catálogo de servicios - Servicios continúan operando normalmente durante transición - Clientes tienen failover automático entre múltiples nodos

Nivel de Tolerancia a Fallos Esperado

Tolerancia a Fallos de MetaNameNode: - Puede tolerar fallo de hasta 2 nodos (de 3 totales) - Con 2 réplicas activas: continúa procesando operaciones, mantiene consenso, realiza elecciones - Con 1 réplica activa: continúa procesando operaciones, no es posible lograr consenso ni realizar elecciones producto a la ausencia de nodos

Tolerancia a Fallos de DataNode: - Puede tolerar fallo simultáneo de hasta 2 DataNodes (de 3 réplicas por archivo) - Con 2 réplicas activas: continúa sirviendo solicitudes de lectura, re-replica automáticamente, mantiene disponibilidad - Con 1 réplica activa: continúa sirviendo solicitudes de lectura, aunque ya no es posible re-replicar automáticamente producto a la ausencia de nodos

Tolerancia a Fallos de Registry: - Puede tolerar fallo de hasta 2 nodos (de 3 totales) - Con 2 réplicas activas: continúa descubrimiento de servicios, mantiene consenso, realiza elecciones - Con 1 réplica activa: continúa descubrimiento de servicios, no mantiene consenso ni realiza elecciones

Fallos Parciales

Nodos Caídos Temporalmente: - Nodos que se reinician se registran automáticamente al volver en línea - DataNodes que se recuperan pueden contener archivos re-replicados durante ausencia - Sistema detecta y maneja archivos duplicados - Reintegración automática sin intervención manual

Nodos Nuevos que se Incorporan: - Nuevos DataNodes se registran automáticamente con MetaNameNode al iniciar - MetaNameNode comienza a asignar nuevas réplicas a DataNodes nuevos - Integración gradual en distribución de carga - No requiere reinicio del sistema

Drenaje Controlado de Nodos: - Mecanismo de drenaje para remover DataNodes de forma segura - DataNode se marca para drenaje (evita nuevas asignaciones) - Todos los archivos se re-repican a otros DataNodes - Una vez completado, DataNode puede removerse sin pérdida de datos - Permite mantenimiento planificado sin interrupciones

Recuperación Automática: - Procesos de re-replicación se ejecutan automáticamente en segundo plano - Sistema restaura automáticamente nivel de replicación deseado - Clientes experimentan mínima interrupción durante recuperación - No requiere intervención manual para mayoría de fallos

8. Seguridad

Seguridad en la Comunicación

Protocolo de Comunicación: - Actualmente: HTTP para todas las comunicaciones - **Recomendación para producción:** migración a HTTPS para cifrado - Uso de certificados TLS para autenticación de servicios - Validación de certificados para prevenir ataques man-in-the-middle

Comunicación en Red Privada: - Servicios se comunican a través de red overlay de Docker Swarm (**tbfs_net**) - Red distribuida que funciona en todos los nodos del swarm - Aislamiento de red pública - Prevención de acceso no autorizado desde fuera de la red - Control de tráfico mediante configuración de red Docker Swarm

Validación de Datos: - Validación de tipos mediante modelos Pydantic - Validación de formatos (ej: hashes SHA-256) - Sanitización de entradas para prevenir inyección de datos - Validación de tamaños de archivo y límites de recursos

Seguridad en el Diseño

Separación de Responsabilidades: - Arquitectura separa MetaNameNode (metadatos) de DataNodes (almacenamiento) - Limitación del alcance de un compromiso potencial - DataNode comprometido no puede acceder directamente a metadatos - MetaNameNode comprometido no puede acceder directamente a archivos físicos

Aislamiento de Contenedores: - Cada servicio ejecuta en contenedor Docker aislado dentro del swarm - Límites de recursos (CPU, memoria) por contenedor - Aislamiento de sistemas de archivos - Prevención de interferencia entre servicios - Distribución automática de contenedores en nodos del swarm

Gestión de Volúmenes: - Volúmenes persistentes aislados por servicio - Cada MetaNameNode tiene su propio volumen de base de datos - Cada DataNode tiene su propio volumen de almacenamiento - Prevención de acceso cruzado entre volúmenes

Validación de Integridad: - Archivos identificados mediante hashes SHA-256 - Detección de corrupción de datos - Verificación de integridad al leer archivos - Prevención de modificación no autorizada de archivos

Autorización y Autenticación

Estado Actual: - Sistema no implementa autenticación ni autorización - Todos los servicios accesibles sin credenciales - Apropiado para entorno de desarrollo o red privada confiable

Recomendaciones para Producción:

Autenticación de Clientes: - Sistema de autenticación basado en tokens (JWT) - Validación de credenciales antes de permitir operaciones - Gestión de sesiones para clientes autenticados

Autorización Basada en Roles: - Control de acceso basado en roles (RBAC) - Permisos diferenciados para lectura y escritura - Auditoría de operaciones para trazabilidad

Conclusión

Se pretende que el sistema TBFS implemente una arquitectura distribuida robusta que separe efectivamente las responsabilidades de descubrimiento de servicios, gestión de metadatos y almacenamiento físico. La implementación de técnicas de replicación, consenso distribuido y tolerancia a fallos debe proporcionar un sistema altamente disponible y escalable.

El uso de REST para comunicación, Docker Swarm para despliegue distribuido, y algoritmos de consenso tipo Raft para coordinación, resultan en un sistema técnicamente sólido y práctico de mantener. La separación de metadatos y almacenamiento físico permite optimizaciones independientes y escalabilidad horizontal mediante adición de nodos al swarm.

Las áreas identificadas para mejora futura incluyen la implementación de seguridad más robusta (autenticación, autorización, cifrado) y optimizaciones adicionales para rendimiento en cargas de trabajo de gran escala.